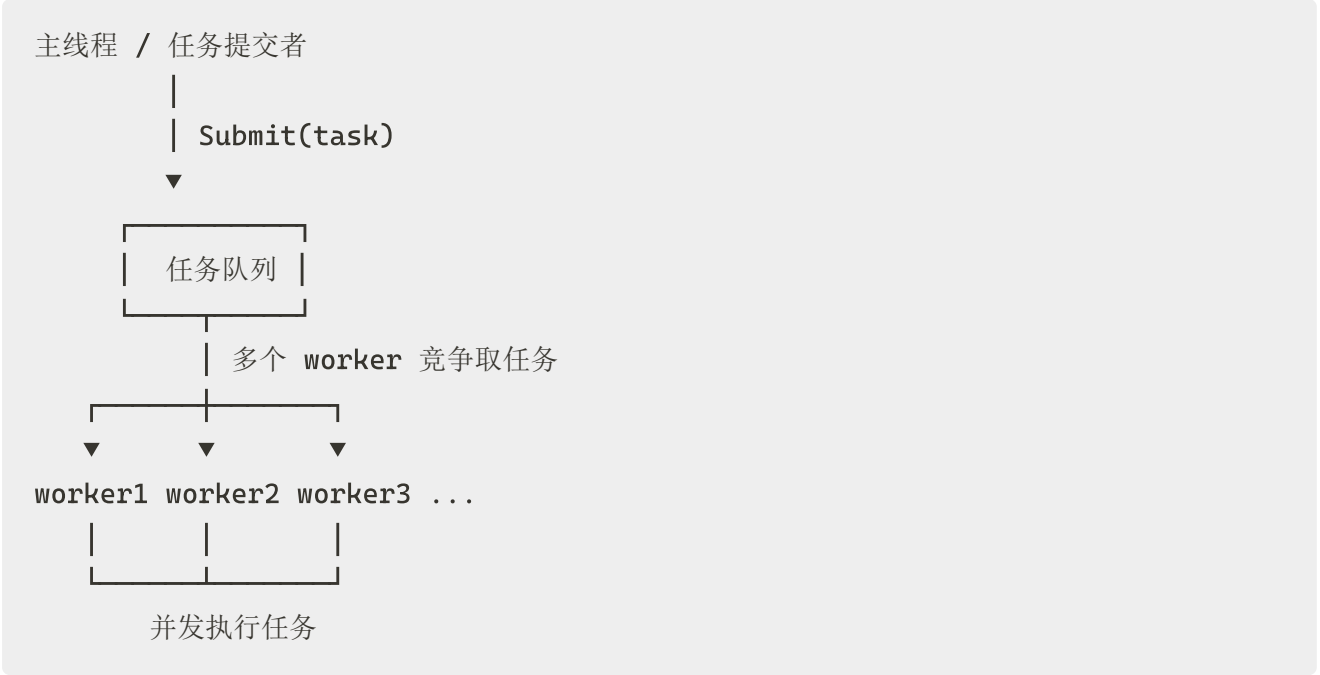
2026-8-21 Linux线程--线程池

- 1. 从“一任务一线程”到线程池
  - 1.1 线程池与生产者消费者模型
  - 1.2 线程池解决什么问题
  - 1.3 线程数量不是越多越好
- 2. 线程池的核心组成
  - 2.1 任务对象
  - 2.2 任务队列
  - 2.3 工作线程集合
  - 2.4 条件变量和停止标志
- 3. 任务在线程池中的流转
  - 3.1 提交任务
  - 3.2 worker 领取任务
  - 3.3 为什么必须在锁外执行任务
  - 3.4 通知的丢失为什么不会造成问题
- 4. 线程池实现
  - 4.1 基本类结构
  - 4.2 构造与线程创建
  - 4.3 提交任务
  - 4.4 worker 主循环
  - 4.5 关闭与回收线程
  - 4.6 使用示例
- 5. 线程池的关键同步规则
  - 5.1 哪些状态必须加锁
  - 5.2 为什么提交和关闭必须互斥
  - 5.3 为什么 worker 等待必须使用 while
  - 5.4 为什么关闭时要使用 broadcast
  - 5.5 析构顺序不能颠倒
- 6. 线程池的边界与常见问题
  - 6.1 线程池不能解决所有延迟问题
  - 6.2 队列积压需要监控
  - 6.3 优雅关闭与立即关闭
- 7. 线程池的核心结论

1. 从“一任务一线程”到线程池

如果每来一个任务就调用一次 `pthread_create`，任务结束后再销毁线程，就会不断付出线程创建、栈空间分配和上下文切换的成本。当任务数量很大或到达速度不均匀时，系统线程数也可能失控。

线程池预先创建固定数量的工作线程，让它们反复从共享任务队列中领取任务：



1.1 线程池与生产者消费者模型

线程池本质上是一个典型的生产者消费者模型：

- 主线程或业务线程产生任务，是生产者。
- 工作线程从任务队列取任务，是消费者。
- 任务队列是共享缓冲区。
- 工作线程取出任务后，可以在队列之外并行执行。

主线程负责“产生任务和选择处理方式”，工作线程负责“真正执行任务”。任务的具体内容可以是计算、文件操作、网络请求、日志处理或其他业务函数。

1.2 线程池解决什么问题

线程池的主要价值包括：

- **复用线程**：避免每个任务都创建和销毁线程。
- **限制并发规模**：工作线程数量固定，避免线程无限增加。

- **削峰填谷**：短时间大量任务可以先进入队列，由工作线程逐步处理。
- **提交与执行解耦**：提交任务的线程不必等待任务执行完成。
- **统一管理生命周期**：线程创建、任务分发、停止和回收集中在线程池内部。

线程池并不让每个任务都立刻获得一个线程。任务需要等待空闲 worker；如果任务执行时间很长或队列积压严重，仍需要调整线程数量、优化任务或增加拒绝和限流策略。

1.3 线程数量不是越多越好

工作线程太少时，CPU 或 I/O 能力可能利用不足；线程太多时，会增加上下文切换、栈空间和调度开销。

线程池大小应根据任务类型确定：

- CPU 密集型任务通常接近可用 CPU 核心数。
- 存在大量阻塞 I/O 的任务可能需要更多线程，但要结合连接数、超时和内存限制。
- 无论采用何种经验公式，都应通过实际负载测试调整，而不是固定认为线程越多越快。

2. 线程池的核心组成

2.1 任务对象

线程池需要统一保存不同类型的任务，因此通常使用可调用对象表示任务：

```
using Task = std::function<void()>;
```

任何一个无参数、无返回值的可调用对象都可以包装成任务：

```
pool.Submit([] {
    std::cout << "执行任务" << std::endl;
});
```

如果需要获得任务结果，可以在外层额外封装 `std::future` 和 `std::promise`，但这属于任务返回值管理，不是线程池能够调度任务的前提。

2.2 任务队列

任务队列保存尚未被 worker 取走的任务：

```
std::queue<Task> tasks_;
```

队列被多个线程共同访问，必须使用互斥锁保护：

```
pthread_mutex_t mutex_;
```

多个提交者可能同时调用 `Submit`，多个 worker 也可能同时等待并领取任务。对 `tasks_` 的 `push`、`front`、`pop` 和 `empty` 检查必须位于同一把锁的保护范围内。

2.3 工作线程集合

工作线程集合保存所有 worker 的线程标识：

```
std::vector<pthread_t> workers_;
```

线程池创建时启动固定数量的 worker。每个 worker 都执行同一个循环：

1. 等待任务队列非空或线程池要求退出。
2. 从队头取走一个任务。
3. 解锁后执行任务。
4. 返回循环，继续等待下一个任务。

2.4 条件变量和停止标志

至少需要一个条件变量表示“可能有新任务”：

```
pthread_cond_t task_available_;
```

还需要一个停止标志：

```
bool stopping_ = false;
```

停止标志和任务队列都受同一把互斥锁保护。关闭线程池时设置停止标志，再广播唤醒所有 worker，避免仍有 worker 永久等待。

3. 任务在线程池中的流转

3.1 提交任务

`Submit` 的职责是把任务安全地放入队列，并唤醒一个等待中的 worker：

```
加锁
  ↓
检查线程池是否已经停止
  ↓
将任务放入队尾
  ↓
signal(task_available)
  ↓
解锁
```

提交者不在锁内执行任务，只完成入队和通知。任务的执行时间不会影响提交接口的锁占用时间。

3.2 worker 领取任务

worker 的主循环可以用以下状态表示：

```
加锁
  ↓
队列为空且未停止？
  ├─ 是：在 task_available 上等待
  └─ 否：继续
  ↓
已经停止且队列为空？
  ├─ 是：解锁并退出 worker
  └─ 否：从队头取出任务
  ↓
解锁
  ↓
执行任务
  ↓
回到循环开头
```

等待条件必须使用 `while` 循环检查。`pthread_cond_wait` 返回只表示 worker 需要重新查看队列，不能直接假定队列已经非空。

3.3 为什么必须在锁外执行任务

worker 取走任务后应立即解锁，再执行任务。如果一直持有队列锁执行任务，其他 worker 无法领取新任务，提交者也无法继续入队，线程池会退化成“一个队列锁串行执行所有任务”。

因此任务队列锁只保护队列状态：

```
锁内：检查队列、取出任务
锁外：执行任务
```

任务执行期间，其他 worker 可以并行处理其他任务，主线程也可以继续提交新任务。

3.4 通知的丢失为什么不会造成问题

`pthread_cond_signal` 只唤醒一个等待者；如果发出通知时恰好没有 worker 等待，这次通知可能丢失。

但任务队列是持久保存的共享状态。后来的 worker 在进入循环时会重新检查 `tasks_.empty()`，发现队列非空就直接取任务，不依赖曾经收到过通知。

这说明条件变量负责唤醒，任务队列负责保存事实。正确设计不能只靠通知计数，而要让等待线程检查可重复观察的共享条件。

4. 线程池实现

4.1 基本类结构

下面的实现使用 POSIX 线程、互斥量和条件变量：

```
#pragma once

#include <cstddef>
#include <functional>
#include <pthread.h>
#include <queue>
```

```cpp
#include <stdexcept>
#include <utility>
#include <vector>

class ThreadPool {
public:
    using Task = std::function<void()>;

    explicit ThreadPool(std::size_t thread_count);
    ~ThreadPool();

    ThreadPool(const ThreadPool&) = delete;
    ThreadPool& operator=(const ThreadPool&) = delete;

    void Submit(Task task);
    void Shutdown();

private:
    static void* WorkerEntry(void* arg);
    void WorkerLoop();

    std::queue<Task> tasks_;
    std::vector<pthread_t> workers_;

    pthread_mutex_t mutex_;
    pthread_cond_t task_available_;
    bool stopping_ = false;
};
```

任务队列、线程集合、停止标志、互斥量和条件变量共同组成线程池的共享状态。

4.2 构造与线程创建

```cpp
ThreadPool::ThreadPool(std::size_t thread_count)
{
    if (thread_count == 0) {
        throw std::invalid_argument("thread_count must be positive");
    }

    if (pthread_mutex_init(&mutex_, nullptr) != 0) {
        throw std::runtime_error("pthread_mutex_init failed");
    }

    if (pthread_cond_init(&task_available_, nullptr) != 0) {
```

```
        pthread_mutex_destroy(&mutex_);
        throw std::runtime_error("pthread_cond_init failed");
    }

    workers_.reserve(thread_count);

    for (std::size_t i = 0; i < thread_count; ++i) {
        pthread_t worker;
        int result = pthread_create(&worker, nullptr, WorkerEntry, this);

        if (result != 0) {
            pthread_mutex_lock(&mutex_);
            stopping_ = true;
            pthread_cond_broadcast(&task_available_);
            pthread_mutex_unlock(&mutex_);

            for (pthread_t created : workers_) {
                pthread_join(created, nullptr);
            }

            pthread_cond_destroy(&task_available_);
            pthread_mutex_destroy(&mutex_);
            throw std::runtime_error("pthread_create failed");
        }

        workers_.push_back(worker);
    }
}
```

每个 worker 都会收到 `this`，因此能够访问同一个任务队列和同步对象。构造函数必须在返回前完成所有线程的创建，这样对象一旦可用，工作线程就已经准备就绪。

4.3 提交任务

```
void ThreadPool::Submit(Task task)
{
    pthread_mutex_lock(&mutex_);

    if (stopping_) {
        pthread_mutex_unlock(&mutex_);
        throw std::runtime_error("thread pool is stopping");
    }

    tasks_.push(std::move(task));
```

```
    pthread_cond_signal(&task_available_);

    pthread_mutex_unlock(&mutex_);
}
```

如果已经请求关闭线程池，新的任务会被拒绝。否则任务入队后通知一个 worker，让任务尽快被处理。

4.4 worker 主循环

```
void* ThreadPool::WorkerEntry(void* arg)
{
    auto* pool = static_cast<ThreadPool*>(arg);
    pool->WorkerLoop();
    return nullptr;
}

void ThreadPool::WorkerLoop()
{
    while (true) {
        pthread_mutex_lock(&mutex_);

        while (tasks_.empty() && !stopping_) {
            pthread_cond_wait(&task_available_, &mutex_);
        }

        if (stopping_ && tasks_.empty()) {
            pthread_mutex_unlock(&mutex_);
            return;
        }

        Task task = std::move(tasks_.front());
        tasks_.pop();
        pthread_mutex_unlock(&mutex_);

        try {
            task();
        } catch (...) {
            // 单个任务失败不能终止整个 worker 循环。
        }
    }
}
```

worker 在锁内只完成“检查、取出”两步，随后在锁外执行任务。这样多个 worker 可以真正并发地处理不同任务。

任务中的异常必须由线程池捕获。异常如果逃出线程入口函数，可能导致整个进程终止，并使对应 worker 无法继续服务。

4.5 关闭与回收线程

```
void ThreadPool::Shutdown()
{
    pthread_mutex_lock(&mutex_);

    if (stopping_) {
        pthread_mutex_unlock(&mutex_);
        return;
    }

    stopping_ = true;
    pthread_cond_broadcast(&task_available_);
    pthread_mutex_unlock(&mutex_);

    for (pthread_t worker : workers_) {
        pthread_join(worker, nullptr);
    }
    workers_.clear();
}

ThreadPool::~ThreadPool()
{
    Shutdown();
    pthread_cond_destroy(&task_available_);
    pthread_mutex_destroy(&mutex_);
}
```

这里采用“优雅关闭”策略：

1. 设置 `stopping_ = true`，禁止继续提交新任务。
2. 广播唤醒所有正在等待的 worker。
3. worker 发现停止标志后，不会立刻丢弃队列中已有的任务，而是继续执行到队列为空。
4. worker 只有在“已经停止且队列为空”时才退出。
5. 主线程通过 `pthread_join` 回收所有 worker。

6. 确认全部 worker 退出后，销毁条件变量和互斥量。

4.6 使用示例

```
#include <chrono>
#include <iostream>
#include <thread>
#include "ThreadPool.hpp"

int main()
{
    ThreadPool pool(4);

    for (int i = 0; i < 10; ++i) {
        pool.Submit([i] {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            std::cout << "task " << i
                      << " on thread " << std::this_thread::get_id()
                      << std::endl;
        });
    }

    pool.Shutdown();
    return 0;
}
```

10 个任务会由 4 个 worker 分批领取。即使提交循环很快结束，线程池也会先完成队列中剩余任务，再让 worker 退出。

5. 线程池的关键同步规则

5.1 哪些状态必须加锁

以下状态都会被多个线程访问：

- `tasks_` 任务队列。
- `stopping_` 停止标志。
- 工作线程判断队列是否为空的结果。

它们的读取和修改必须使用同一把 `mutex_`。只在 `Submit` 中加锁、worker 却不加锁，或者反过来，都会形成数据竞争。

5.2 为什么提交和关闭必须互斥

如果线程池正在关闭，却仍允许新任务加入队列，就可能出现 worker 全部退出后队列里还有任务的情况。

`Submit` 与 `Shutdown` 通过同一把锁检查 `stopping_`：

- 关闭开始后，后续 `Submit` 失败并拒绝任务。
- 已经位于队列中的任务仍由 worker 继续处理。

5.3 为什么 worker 等待必须使用 `while`

worker 被唤醒后，队列可能再次被其他 worker 取空。伪唤醒也可能在没有新任务时返回。

因此等待条件必须是：

```
while (tasks_.empty() && !stopping_) {
    pthread_cond_wait(&task_available_, &mutex_);
}
```

只有这样，worker 才会在条件真正满足后继续执行。

5.4 为什么关闭时要使用 `broadcast`

`pthread_cond_signal` 只通知一个等待者。关闭线程池时需要唤醒所有正在等待任务的 worker，因此要使用：

```
pthread_cond_broadcast(&task_available_);
```

每个 worker 被唤醒后重新取得锁，发现停止标志后依次检查队列并按规则退出。

5.5 析构顺序不能颠倒

必须先执行 `Shutdown` 并 `join` 所有 worker，再销毁互斥量和条件变量。

如果先销毁同步对象，仍在运行的 worker 访问它们会造成未定义行为；如果忘记 `join`，线程也可能在对象析构后继续访问已经释放的成员。

6. 线程池的边界与常见问题

6.1 线程池不能解决所有延迟问题

线程池只复用工作线程，不能让任务自动变快。大量 CPU 密集任务仍受 CPU 数量限制，大量阻塞 I/O 任务仍可能耗尽线程和连接资源。

工作线程也不能长时间执行没有终点的任务，否则其他任务只能排队等待。

6.2 队列积压需要监控

任务生产速度长期高于消费速度时，任务队列会持续增长。即使线程池本身逻辑正确，也可能出现内存占用升高和任务延迟增加。

实际系统应监控队列长度、任务等待时间和 worker 利用率，并根据业务需要设置容量上限、拒绝策略或背压。

6.3 优雅关闭与立即关闭

本节实现的是优雅关闭：停止接收新任务，但继续执行队列中已有的任务。

如果业务要求立即停止，可以在关闭时清空任务队列，或为任务增加取消标志。但这种策略会改变任务完成语义，必须在提交者、worker 和销毁逻辑之间统一约定。

7. 线程池的核心结论

- 线程池预先创建固定数量的 worker，并通过共享任务队列分发任务。
- 主线程是任务生产者，worker 是任务消费者，整体结构对应生产者消费者模型。
- 任务队列和停止标志必须由互斥量保护；worker 等待使用条件变量和 `while` 条件检查。
- worker 必须在取得任务后先解锁，再执行任务，否则所有任务会被队列锁串行化。
- `Submit` 负责入队并唤醒 worker，`Shutdown` 负责停止接收、广播唤醒和回收线程。
- 优雅关闭时，worker 应处理完已有任务后再退出。
- 必须先 `join` 所有 worker，再销毁互斥量和条件变量。